

4160 – Distributed and Parallel Computing

P5 – Performance Analysis



CUHK

Performance Models or Tools

- Determine when you shall stop optimizing
 - Motivate if you shall change the algorithm instead
- Identify Performance Bottleneck
 - It is compute-bound or memory-bound?
- Predict performance on another machine
 - E.g., whether things will get faster if you buy another machine
- One great free tool is perf
 - <https://perf.wiki.kernel.org/index.php/Tutorial>
 - <https://www.brendangregg.com/perf.html>

Memory-Bound vs Compute-Bound

- Consider NVIDIA GTX 680
- Max GPU Memory Bandwidth
 - 192GB/sec
 - = $192 * 10^9$ bytes per second
 - = $48 * 10^9$ floating-point (single precision; so 4-bytes) can be loaded per second
- Max Compute
 - 3090 GFLOPS = $3090 * 10^9$



Performance Model

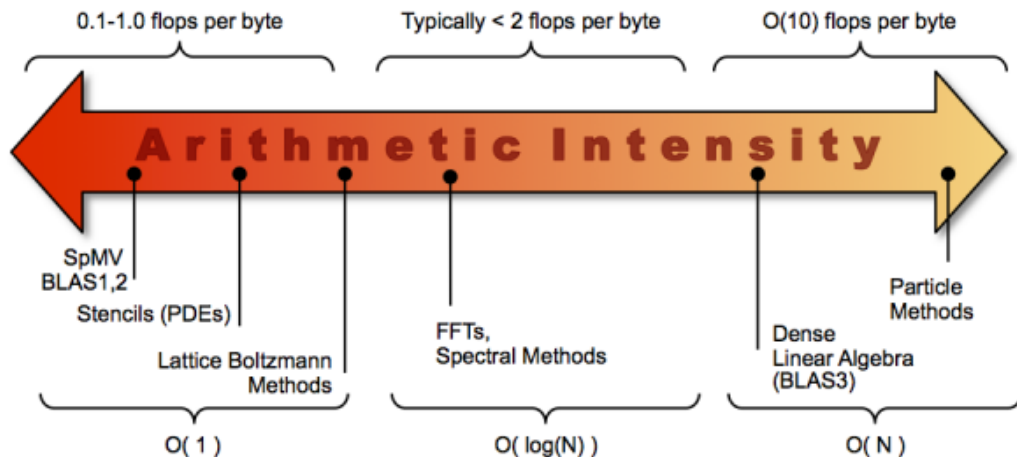
- Affected by many factors
 - Flop/s
 - Cache bandwidth
 - DRAM bandwidth
 - ...

Operational Intensity (FLOPs/Byte)

- For each byte of data that you transferred between memory and cache
 - How much compute (operation) you will work on it?
- E.g., Matrix Multiplication $c[i] += a[j] * b[k]$
 - Requires
 - 2 data accesses: $a[j]$ and $b[k]$ = 8 bytes
 - Access to $c[i]$ negligible as it is in the cache (only transferred back to the memory once as the final result)
 - 2 FLOPs : + and *
 - $2 / 8 = 0.25$ FLOPs/byte

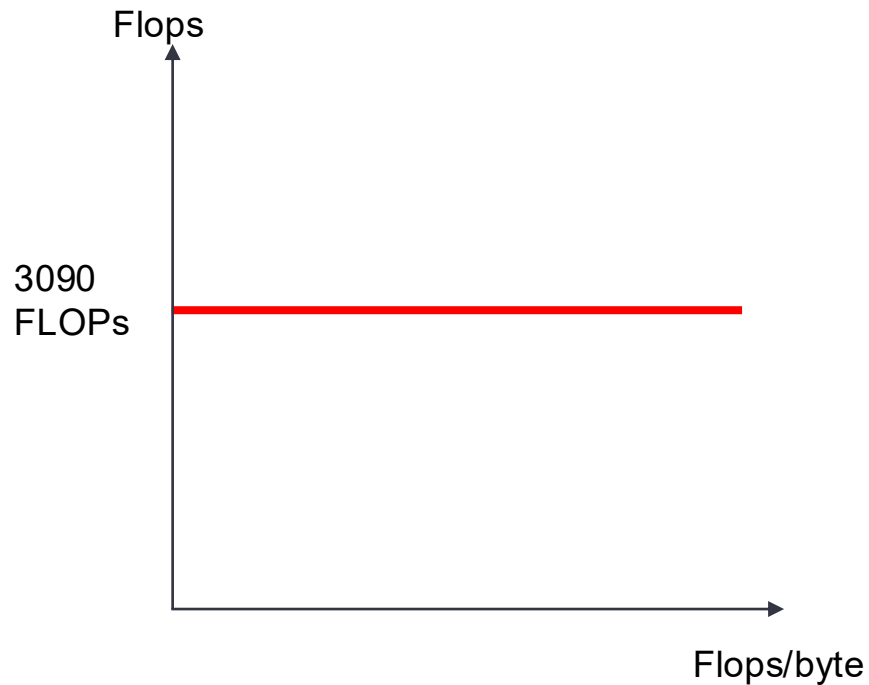
Arithmetic Intensity

- Similar to OI
- For each byte of data that you **transferred between cache and core**
 - How much compute (operation) you will work on it?



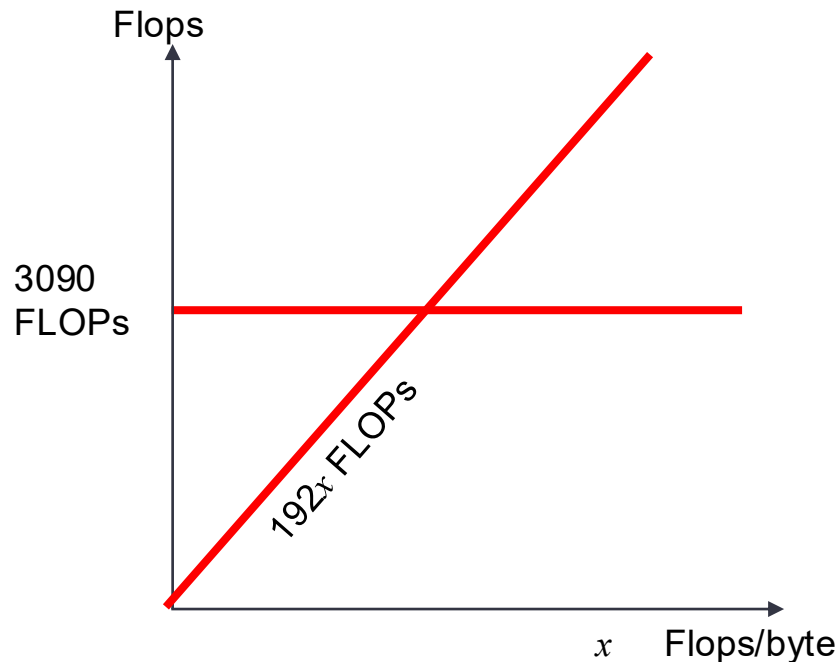
Roofline of your device

- E.g., GTX 680 can do
 - **max 3090GFlop/s**
 - Max memory bandwidth: 192GB/s



Roofline of your device

- E.g., GTX 680 can do
 - max 3090GFlop/s
 - Device's compute-bound
 - **Max memory bandwidth: 192GB/s**
- Given a kernel with an $AI=x$ FLOPs/byte
 - Assume your program is able to consume the maximum 192GB/s
 - i.e., achieve:
 - x FLOPs/byte * 192GB/s
 - Device's memory-bound
 - 192x FLOPs



Optimizing your kernel

- My Matrix Multiplication kernel

- A tool tells my kernel's AI = 0.25 FLOPs/byte, consumed memory bandwidth is 10GB/s, then, my program kernel is:

- $\text{FLOP/s} = \text{AI}_{\text{FLOPs/byte}} * \text{consumed bandwidth}_{\text{GB/s}}$

- To consume more bandwidth

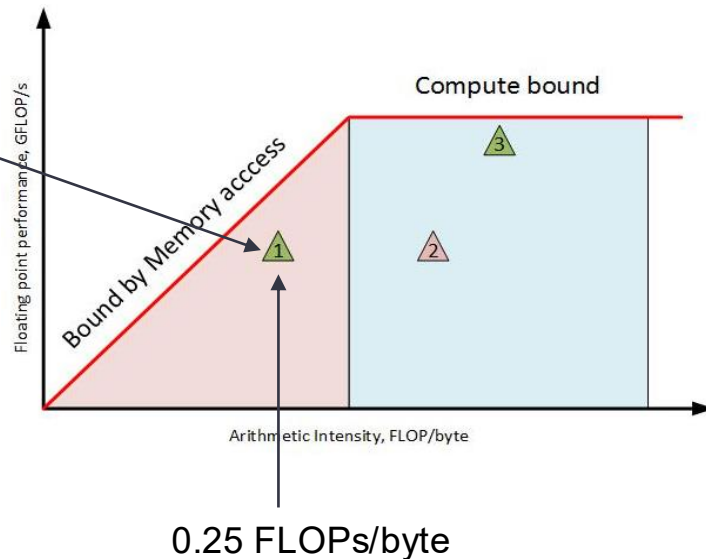
- Bring in more data for processing (e.g., increase # of threads to access more data in parallel)

- To improve AI:

- $\text{AI} = \frac{\text{\# of operations}}{\text{\# of bytes load from memory}}$

- If two reads on a[71] are far apart (cache miss!)

- Try to rearrange your code so that a[71] is still in the cache in the 2nd read
- cache hit reduces the denominator



Memory-bound vs Compute-bound

- If your problem nature is data-intensive (e.g., database SUM query) and your implementation is memory-bound
 - It is a positive result
 - Mem bandwidth is decided by processor memory controller
 - GPU: use a GPU with more mem bandwidth
 - CPU: use NUMA or CXL memory expander (new!)
- If your problem nature is compute-intensive, i.e., high AI (e.g., simulation, dense algebra) and your implementation is compute-bound
 - It is a positive result
 - Find a more powerful GPU

	GTX 1080	GTX 1070	GTX 1060	GTX 1050
	1080 Ti / 1080	1070 Ti / 1070	6GB / 3GB	1050 Ti / 1050
GPU ENGINE SPECS				
CUDA Cores	3584 / 2560	2432 / 1920	1280 / 1152	768 / 640
Base Clock [MHz]	1480 / 1607	1607 / 1506	1506	1290 / 1354
Boost Clock [MHz]	1582 / 1733	1683 / 1683	1708	1392 / 1455
MEMORY SPEC				
Memory Speed	11 Gbps / 10 Gbps	8.0 Gbps	8.0 Gbps	7.0 Gbps
Standard Memory Config	11 GB GDDR5X / 8 GB GDDR5X	8 GB GDDR5	6 GB GDDR5 / 3 GB GDDR5	4 GB GDDR5 / 2 GB GDDR5
Memory Interface Width	352-bit / 256-bit	256-bit	192-bit	128-bit
Memory Bandwidth (GB/sec)	484 / 320	256	192	112



GTX680 Vs H200

- 0.192TB/s vs 4.9TB/s



Reference

- <https://perf.wiki.kernel.org/index.php/Tutorial>
- <https://crd.lbl.gov/assets/Uploads/ECP18-Roofline-1-intro.pdf>
- <https://www.youtube.com/watch?v=h2QEM1HpFgg>
- <https://insidehpc.com/2019/02/improving-hpc-performance-with-the-roofline-model/>